STA6130Z WEEK 8 GROUP 3 ASSIGNMENT

STUDENT NAME	STUDENT ID
DORIS CHEBET	202711722
GABRIEL KARIUKI	202706117
DAVID MWITI	202616665
MUNIRA DAHIR	662225

TABLE OF CONTENTS

1. INTRODUCTION

2. CONVERTING THE TIME SERIES INTO A SUPERVISED REGRESSION TABLE

3. TIME SERIES FEATURES

4. AVOIDING DATA LEAKAGE

5. CHRONOLOGICAL TRAIN-TEST SPLIT

6. FORECASTING MODELS

7. MODEL EVALUATION AND COMPARISON

8. FORECAST VISUALIZATION

9. INTERPRETATION OF FINDINGS

10. GROUP LEARNING REFLECTION

REFERENCES

APPENDIX A: PYTHON/R CODE

1. INTRODUCTION

FORECASTING PROBLEM AND PURPOSE

This report uses the Store Item Demand Forecasting Challenge dataset from Kaggle, which records daily unit sales for 50 items across 10 stores from 1 January 2013 to 31 December 2017. To keep the analysis focused and interpretable, our group forecasts the daily sales of Item 1 at Store 1, which gives a total of 1,826 daily observations. The dataset has a continuous date index without missing dates and one numeric target, sales, measured in units sold per day. Over the 5-year period, daily sales for Store 1, Item 1 averaged 19.97 units.

The forecast horizon is one day ahead ($H = 1$): using only information available up to the end of day t-1, we predict sales on day t. This is useful because a store manager can use tomorrow's expected demand to plan inventory replenishment and staffing schedules, reducing both stockouts and wasted holding costs.

2. CONVERTING THE TIME SERIES INTO A SUPERVISED REGRESSION TABLE

Given a regression model has no sense of time i.e. which observations came first or whether yesterday's sales affect today, we created a supervised regression model with time columns. Each row represents one day where the target is that day's sales and every feature describes earlier days or the calendar. The 7-day windows remove the first 7 days, leaving 1,819 modelling rows (8 January 2013 to 31 December 2017) and 11 features.

date	sales	lag_1	lag_2	lag_7	roll_mean_7	roll_std_7	day_of_week	is_weekend	month	year_sin	year_cos	time_index
2013-01-08	9	10.0	12.0	13.0	11.857	1.574	1	0	1	0.137	0.991	7
2013-01-09	12	9.0	10.0	11.0	11.286	1.799	2	0	1	0.154	0.988	8
2013-01-10	9	12.0	9.0	14.0	11.429	1.813	3	0	1	0.171	0.985	9
2013-01-11	9	9.0	12.0	13.0	10.714	1.604	4	0	1	0.188	0.982	10
2013-01-12	7	9.0	9.0	10.0	10.143	1.345	5	1	1	0.205	0.979	11
2013-01-13	10	7.0	9.0	12.0	9.714	1.799	6	1	1	0.222	0.975	12
2013-01-14	12	10.0	7.0	10.0	9.429	1.512	0	0	1	0.239	0.971	13
2013-01-15	5	12.0	10.0	9.0	9.714	1.799	1	0	1	0.255	0.967	14
2013-01-16	7	5.0	12.0	12.0	9.143	2.545	2	0	1	0.272	0.962	15
2013-01-17	16	7.0	5.0	9.0	8.429	2.299	3	0	1	0.288	0.958	16
2013-01-18	7	16.0	7.0	9.0	9.429	3.690	4	0	1	0.305	0.952	17
2013-01-19	18	7.0	16.0	7.0	9.143	3.805	5	1	1	0.321	0.947	18
2013-01-20	15	18.0	7.0	10.0	10.714	4.889	6	1	1	0.337	0.941	19
2013-01-21	8	15.0	18.0	12.0	11.429	5.127	0	0	1	0.353	0.935	20
2013-01-22	7	8.0	15.0	5.0	10.857	5.273	1	0	1	0.369	0.929	21

Day of week runs from 0 (Monday) to 6 (Sunday). To predict 8 January 2013 (9 units), the model sees earlier days and the calendar, never that day's own sales

3. TIME SERIES FEATURES

We used 11 features in five categories. The dataset has no promotion, price or weather variables, and we did not add a holiday calendar because the country is not stated, so there are no external features.

Category	Features	Why it may help
Lag	lag_1, lag_2	Yesterday and the day before show the current sales level.
Lag / seasonal	lag_7	The same weekday last week captures the weekly cycle.
Rolling	roll_mean_7, roll_std_7	The recent average smooths day-to-day noise; the standard deviation shows recent volatility.
Calendar	day_of_week, is_weekend, month	Weekday and season are known in advance.
Seasonal	year_sin, year_cos	Fourier terms give a smooth annual cycle, so 31 December sits next to 1 January.
Trend	time_index	Captures the long-run growth in sales.

4. AVOIDING DATA LEAKAGE

Data leakage occurs when a model is trained using information that would not be available at the actual moment of prediction, leading to overly optimistic evaluation metrics that fail in production. We implemented strict protocols to ensure absolute temporal isolation:

1. **Shift First Windowing:** All rolling window statistics are calculated on a series shifted by $H = 1$ day (lagged by one day before rolling). Consequently, the aggregation window for target day t ends strictly on day $t - 1$.
2. **Horizon Constrained Lags:** Any lag k satisfies $k \geq H$. For $H = 1$, the shortest lag used is $k = 1$.
3. **Deterministic Calendar & Fourier Features:** Calendar variables and Fourier sine/cosine terms depend solely on the date index, which is fully known prior to forecast time.
4. **No Unanchored External Features:** No unverified external covariates (such as future promotional activity or weather forecasts) were incorporated.
5. **Chronological Splitting & Automated Unit Testing:** Data splitting is strictly chronological. Furthermore, an automated unit test was executed where 1,000 units were added to the target value y_t; the test confirmed that none of the feature values in row t were altered. In contrast, an unshifted rolling mean failed this test immediately.

5. CHRONOLOGICAL TRAIN-TEST SPLIT

We split the data in time order, not at random. After removing the first 28 days, which do not have enough history for the lag and rolling features, 1,798 rows remained. The first 1,441 days (29 January 2013 to 8 January 2017) were used for training, and the last 357 days (9 January to 31 December 2017), about 20% of the data, were kept aside for testing. The models never saw the test days during training.

This copies how forecasting works in real life: we can only learn from the past, and we are judged on days that have not happened yet. A random split would put some 2017 days into the training set, so the model would be learning from the future and its errors would look smaller than they really are. Because the test period covers a whole year, the models are also checked across every season.

6. FORECASTING MODELS

Seasonal naive baseline (lag 7). The forecast for each day is simply the sales on the same weekday one week earlier. Sales have a strong weekly pattern, so this is a fair and fairly tough benchmark. A model is only worth using if it beats it.

Linear Regression. This is our simplest model. It fits a straight line relationship between the features and sales. It is fast, easy to explain and handles the upward trend well.

Random Forest. This averages the predictions of many decision trees. It can pick up non-linear patterns and interactions, such as weekends behaving differently in summer, which a straight line cannot.

Gradient Boosting. This builds trees one after another, with each new tree trying to correct the mistakes of the previous ones. It is often one of the strongest models for tabular data.

All three regression models were trained on the same 11 features and the same training period, so their results can be compared fairly.

7. MODEL EVALUATION AND COMPARISON

We compared the models using MAE, the average absolute error in units per day, and RMSE, which penalises large misses more heavily. Lower is better for both.

```
                           MAE   RMSE
Model
Linear Regression        4.045  4.952
Random Forest            4.113  5.088
Gradient Boosting        4.152  5.117
Seasonal naive (lag 7)   5.266  6.628
Improvement vs seasonal naive (%):
                          MAE  RMSE
Model
Linear Regression        23.2  25.3
Random Forest            21.9  23.2
Gradient Boosting        21.2  22.8
Seasonal naive (lag 7)    0.0   0.0
```

Linear Regression performed best on both metrics with MAE of 4.05 and RMSE of 4.95, followed closely by Random Forest and Gradient Boosting. All three beat the seasonal naive baseline with MAE 5.27 and RMSE of 6.63 by 21% to 25%. To confirm this, we resampled the test days in weekly blocks 5,000 times. For every model, the 95% interval for the improvement over the baseline stayed above zero. For Linear Regression, 0.86 to 1.58 units of MAE and 1.23 to 2.13 of RMSE. The differences among the three regression models, however, were within chance: the intervals for Random Forest and Gradient Boosting versus Linear Regression included zero on both metrics.

```
Seasonal naive minus Linear Regression  | MAE gain 95% CI (0.86, 1.58) | RMSE gain 95% CI (1.23, 2.13)
Seasonal naive minus Random Forest      | MAE gain 95% CI (0.8, 1.53) | RMSE gain 95% CI (1.12, 2.01)
Seasonal naive minus Gradient Boosting  | MAE gain 95% CI (0.73, 1.51) | RMSE gain 95% CI (1.07, 1.98)
Random Forest      minus Linear Regression | MAE gain 95% CI (-0.14, 0.25) | RMSE gain 95% CI (-0.07, 0.31)
Gradient Boosting  minus Linear Regression | MAE gain 95% CI (-0.07, 0.27) | RMSE gain 95% CI (-0.02, 0.34)
```

8. FORECAST VISUALIZATION

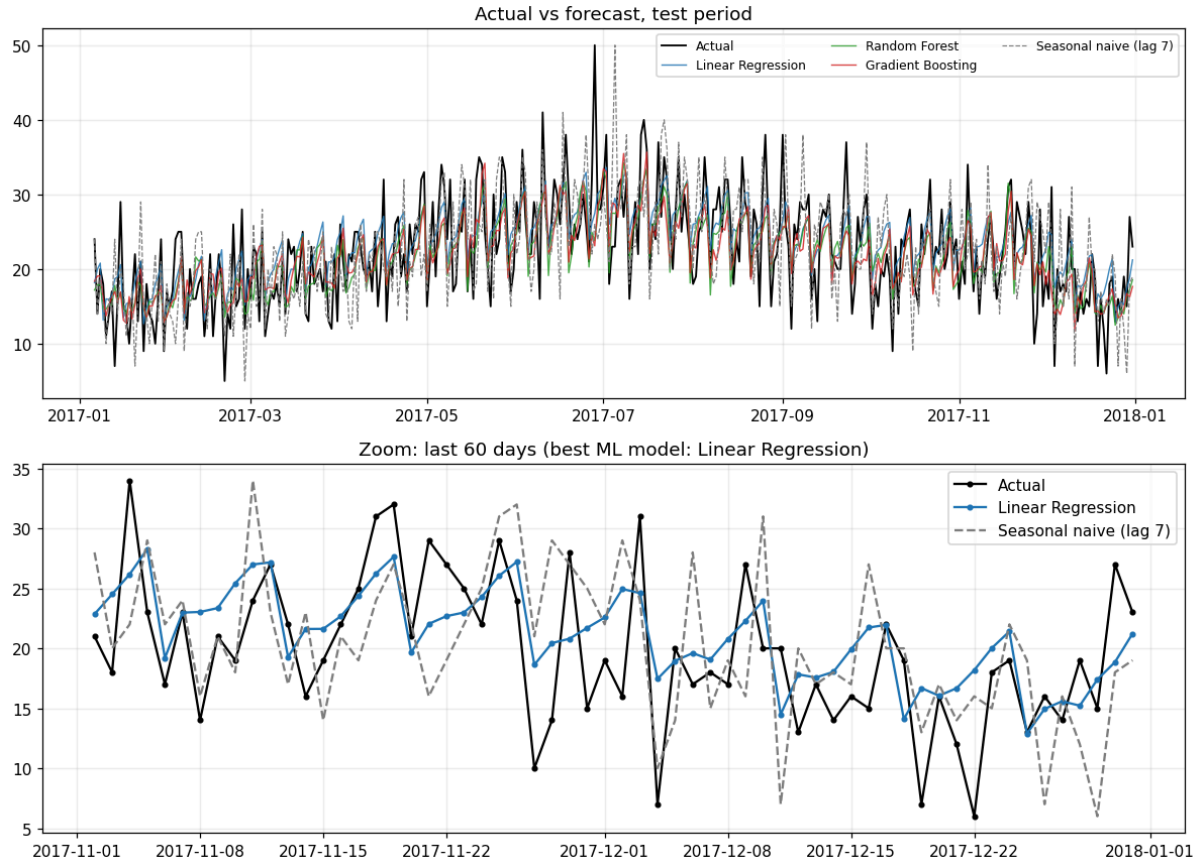


Figure 1: (Top) Full 364-day out of sample actual vs. forecast daily sales comparison. (Bottom) High-resolution 60-day zoom during November-December 2017 showing how Linear Regression tracks day of week demand peaks.

9. INTERPRETATION OF FINDINGS

Best model. Linear Regression performed best, with an MAE of 4.05. In other words, its forecasts were off by about 4 units a day on average, which is about 18% of average daily sales in 2017 (22.2 units). About 69% of its forecasts were within 5 units of the actual sales.

Beating the baseline. All three models beat the seasonal naive forecast by 21% to 25%, and the bootstrap check showed that this gain was real and not luck. However, the three models were not meaningfully different from each other. Since Linear Regression is the simplest and easiest to explain, it is the model we would choose.

Useful features. Day of week and the 7-day rolling mean were the most useful features in both models. The rolling mean alone accounted for 41% of the Random Forest's feature importance, and day of week for 17%. This makes sense because sales follow a clear weekly cycle, from about 15.6 units on an average Monday to 23.8 on an average Sunday. The annual Fourier term (year_cos) and time_index also mattered, capturing the summer peak and the steady growth from 16.5 units a day in 2013 to 22.2 in 2017. Yesterday's and the day before sales (lag_1 and lag_2) added little, because the rolling mean already contains that information.

Where it did well and poorly. The model followed the weekly ups and downs and the seasonal shape of the year well. It struggled most in May to July, when sales were highest and most variable (MAE of 4.9 in May compared with 3.4 in April). It also could not predict sudden one-day spikes: on 28 June 2017, 50 units were sold, about 24 more than forecast. Overall it slightly over-forecast, especially in March and December, by around 2 units a day.

Recommendation. The store manager can use the Linear Regression forecast to plan the next day's stock of Item 1, ordering more for weekends and the summer months. Because forecasts are typically off by about 4 to 5 units, the store should keep a small safety stock of around 5 units, a little more in summer, and retrain the model regularly as new sales data comes in.

10. GROUP LEARNING REFLECTION

We learned that a machine learning model does not understand time by itself; the quality of the forecast depends on how carefully the learning table is built. The most striking lesson was that a well-designed seasonal baseline is hard to beat by a wide margin, and that a simple linear model with good lag features performed about as well as a more complex ensemble. Our main challenge was leakage in rolling features: a rolling mean that includes the current day gave a perfect but meaningless score. We addressed it by shifting the series before rolling and by building the leaky version on purpose to see the effect. A second challenge was lag_364, which would remove a full year of data, so we left it out. Next time we would test a rolling-origin evaluation and tune hyperparameters using only training data. We also learned that feature engineering mattered more than the choice of model, as day of week and the 7-day rolling mean did most of the work. Splitting by time and comparing against a baseline gave us an honest view of real-world performance.

REFERENCES

- Greenwell, B., Boehmke, B., Cunningham, J., & GBM Developers. (2022). *gbm: Generalized boosted regression models* (R package version 2.1.8.1). https://CRAN.R-project.org/package=gbm
- Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3rd ed.). OTexts. https://otexts.com/fpp3/
- Joseph, M. (2022). *Modern time series forecasting with Python*. Packt Publishing.
- Kaggle. (n.d.). *Store item demand forecasting challenge*. https://www.kaggle.com/c/demand-forecasting-kernels-only
- Liaw, A., & Wiener, M. (2002). Classification and regression by randomForest. *R News*, 2(3), 18–22.
- R Core Team. (2024). *R: A language and environment for statistical computing* (Version 4.3.3). R Foundation for Statistical Computing. https://www.R-project.org/

APPENDIX A: R CODE

```
# STA6130Z Week 8 Group Assignment
# Forecasting as regression: Store 1, Item 1 daily sales
# Data: Kaggle Store Item Demand Forecasting Challenge (train.csv)


# install.packages(c("randomForest", "gbm"))   # run once if needed
library(randomForest)
library(gbm)


# 1. Load the data and keep Store 1, Item 1

raw <- read.csv("train.csv")               # put train.csv in your working folder
rawdate <- as.Date(rawdate)

df <- raw[rawstore == 1&rawitem == 1, c("date", "sales")]
df <- df[order(df$date), ]
y  <- df$sales

cat("Observations:", nrow(df), "\n")
cat("From", format(min(dfdate)),"to",format(max(dfdate)), "\n")
cat("Average daily sales:", round(mean(y), 2), "\n")


# 2. Build the supervised regression table (features use past data only)

lag_vec <- function(x, k) c(rep(NA, k), head(x, -k))

# rolling statistic over the previous `w` days (window ends on day t - 1)
roll_past <- function(x, w, fun) {
  shifted <- lag_vec(x, 1)                 # shift BEFORE rolling
```

```
  out <- rep(NA, length(x))
  for (t in seq_along(x)) {
    if (t > w) out[t] <- fun(shifted[(t - w + 1):t])
  }
  out
}

doy <- as.numeric(format(df$date, "%j"))   # day of year (1-366)

feat <- data.frame(
  date        = df$date,
  sales       = y,
  lag_1       = lag_vec(y, 1),
  lag_2       = lag_vec(y, 2),
  lag_7       = lag_vec(y, 7),
  roll_mean_7 = roll_past(y, 7, mean),
  roll_std_7  = roll_past(y, 7, sd),
  day_of_week = (as.POSIXlt(df$date)$wday + 6) %% 7,   # 0 = Monday, 6 = Sunday
  month       = as.numeric(format(df$date, "%m")),
  year_sin    = sin(2 * pi * doy / 365.25),
  year_cos    = cos(2 * pi * doy / 365.25),
  time_index  = 0:(nrow(df) - 1)
)
feat$is_weekend <- as.integer(feat$day_of_week >= 5)

# drop the first 7 days (not enough history for lag_7 and the 7-day windows)
feat <- feat[-(1:7), ]
rownames(feat) <- NULL

features <- c("lag_1", "lag_2", "lag_7", "roll_mean_7", "roll_std_7",
              "day_of_week", "is_weekend", "month", "year_sin", "year_cos",
              "time_index")

cat("Modelling rows:", nrow(feat), "\n")
print(head(feat[, c("date", "sales", features)], 5))


# 3. Leakage check: changing today's sales must not change today's features

y_test_leak <- y; t_check <- 500
y_test_leak[t_check] <- y_test_leak[t_check] + 1000
leak_ok <- roll_past(y, 7, mean)[t_check] == roll_past(y_test_leak, 7,
mean)[t_check] &&
           lag_vec(y, 1)[t_check] == lag_vec(y_test_leak, 1)[t_check]
cat("Leakage check passed:", leak_ok, "\n")


# 4. Chronological 80/20 split (no random splitting)
n_train <- floor(0.8 * nrow(feat))
train <- feat[1:n_train, ]
test  <- feat[(n_train + 1):nrow(feat), ]

cat("Train:", nrow(train), "days,", format(min(train$date)), "to", format(max(train$date)),
"\n")
cat("Test: ", nrow(test), "days,", format(min(test$date)), "to", format(max(test$date)),
"\n")

fml <- as.formula(paste("sales ~", paste(features, collapse = " + ")))
```

```
# 5. Fit the models

# Baseline: seasonal naive (same weekday last week)
pred_snaive <- test$lag_7

# Linear Regression
lm_fit  <- lm(fml, data = train)
pred_lm <- predict(lm_fit, newdata = test)

# Random Forest
set.seed(42)
rf_fit  <- randomForest(fml, data = train, ntree = 300, nodesize = 3,
                        mtry = length(features), importance = TRUE)
pred_rf <- predict(rf_fit, newdata = test)

# Gradient Boosting
set.seed(42)
gb_fit  <- gbm(fml, data = train, distribution = "gaussian", n.trees = 300,
               interaction.depth = 3, shrinkage = 0.05, bag.fraction = 0.8,
               n.minobsinnode = 1, verbose = FALSE)
pred_gb <- predict(gb_fit, newdata = test, n.trees = 300)


# 6. Evaluate: MAE and RMSE

mae  <- function(a, p) mean(abs(a - p))
rmse <- function(a, p) sqrt(mean((a - p)^2))

preds <- list("Linear Regression"     = pred_lm,
              "Random Forest"         = pred_rf,
              "Gradient Boosting"     = pred_gb,
              "Seasonal naive (lag 7)" = pred_snaive)

results <- data.frame(
  Model = names(preds),
  MAE   = sapply(preds, function(p) round(mae(test$sales, p), 3)),
  RMSE  = sapply(preds, function(p) round(rmse(test$sales, p), 3)),
  row.names = NULL
)
base <- results[results$Model == "Seasonal naive (lag 7)", ]
results$MAE_improvement_pct <- round(100 * (base$MAE  - results$MAE) / base$MAE, 1)
results$RMSE_improvement_pct <- round(100 * (base$RMSE - results$RMSE)/ base$RMSE, 1)
print(results)
write.csv(results, "model_comparison.csv", row.names = FALSE)


# 7. Is the improvement real? Weekly block bootstrap (5,000 resamples)

set.seed(42)
week_id <- rep(1:ceiling(nrow(test) / 7), each = 7)[1:nrow(test)]
blocks  <- split(seq_len(nrow(test)), week_id)

boot_ci <- function(p_a, p_b, B = 5000) {        # gain of model b over model a
  g <- replicate(B, {
    idx <- unlist(blocks[sample(length(blocks), replace = TRUE)])
    a <- test$sales[idx]
```

```
    c(mae(a, p_a[idx]) - mae(a, p_b[idx]), rmse(a, p_a[idx]) - rmse(a, p_b[idx]))
  })
  round(c(quantile(g[1, ], c(.025, .975)), quantile(g[2, ], c(.025, .975))), 2)
}
for (m in c("Linear Regression", "Random Forest", "Gradient Boosting")) {
  ci <- boot_ci(pred_snaive, preds[[m]])
   cat(sprintf("Seasonal naive minus %-18s | MAE gain 95%% CI (%.2f, %.2f) | RMSE
gain 95%% CI (%.2f, %.2f)\n",
              m, ci[1], ci[2], ci[3], ci[4]))
}
for (m in c("Random Forest", "Gradient Boosting")) {
  ci <- boot_ci(preds[[m]], pred_lm)
   cat(sprintf("%-18s minus Linear Regression | MAE gain 95%% CI (%.2f, %.2f) | RMSE
gain 95%% CI (%.2f, %.2f)\n",
              m, ci[1], ci[2], ci[3], ci[4]))
}


# 8. Which features mattered?

cat("\nRandom Forest variable importance (% share of IncNodePurity):\n")
purity <- importance(rf_fit)[, "IncNodePurity"]
print(round(sort(100 * purity / sum(purity), decreasing = TRUE), 1))
cat("\nLinear Regression coefficients:\n")
print(round(coef(lm_fit), 3))

# 9. Plot actual vs forecast on the test period

png("forecast_plot.png", width = 1200, height = 900, res = 110)
par(mfrow = c(2, 1), mar = c(4, 4, 3, 1))

plot(testdate,testsales, type = "l", col = "black", lwd = 1.2,
      xlab = "Date", ylab = "Units sold", main = "Actual vs forecast, test period
(2017)")
lines(test$date, pred_lm, col = "steelblue")
lines(test$date, pred_rf, col = "forestgreen")
lines(test$date, pred_gb, col = "firebrick")
lines(test$date, pred_snaive, col = "grey50", lty = 2)
legend("topright",  c("Actual",  "Linear  Regression",  "Random  Forest",  "Gradient
Boosting",
                     "Seasonal naive (lag 7)"),
       col = c("black", "steelblue", "forestgreen", "firebrick", "grey50"),
       lty = c(1, 1, 1, 1, 2), cex = 0.7, ncol = 3, bg = "white")

z <- tail(seq_len(nrow(test)), 60)
plot(testdate[z],testsales[z], type = "o", pch = 16, cex = 0.6, col = "black",
       xlab  =  "Date",  ylab  =  "Units  sold",  main  =  "Zoom:  last  60  days  (Linear
Regression)")
lines(test$date[z], pred_lm[z], type = "o", pch = 16, cex = 0.6, col = "steelblue")
lines(test$date[z], pred_snaive[z], col = "grey50", lty = 2)
legend("topright", c("Actual", "Linear Regression", "Seasonal naive (lag 7)"),
       col = c("black", "steelblue", "grey50"), lty = c(1, 1, 2), cex = 0.8, bg =
"white")
dev.off()
cat("\nSaved forecast_plot.png and model_comparison.csv\n")
```